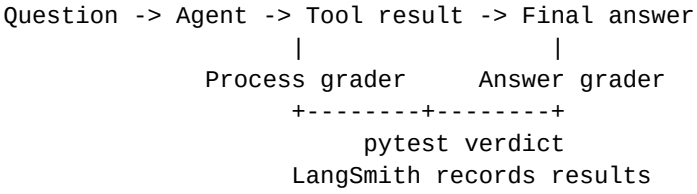
01 / LEARNING NOTES

Your project: the mental model

A beginner guide to everything we have covered, with at least three worked examples for each main topic. Based on your weather-evals project. Prepared 5 October 2026.

The goal: measure whether an AI agent uses its tool correctly and communicates the correct result. You can change a prompt or model and rerun the same questions to compare behavior. These evaluations do not train the model.



Component	Job
LangChain	Builds and runs the agent being tested.
pytest	Discovers tests, executes them, and reports pass/fail.
AgentEvals	Compares actual tool use with an expected path.
OpenEvals	Uses a judge model to assess answer correctness.
LangSmith	Records examples, traces, grading feedback, and experiments.

Example 1: Tokyo: basic facts

The tool returns sunny and 22 degrees Celsius. The expected answer reports those facts.

Example 2: London: facts plus advice

The tool returns rainy and 12 degrees Celsius. The expected answer also recommends an umbrella.

Example 3: Atlantis: unavailable data

The tool returns available=False. The expected answer acknowledges missing data instead of inventing weather.

02 / LEARNING NOTES

The agent and its tool

agent.py combines a model, instructions, and a Python function exposed as a tool. LangChain manages the model/tool conversation.

build_agent() constructs the agent. Its model comes from AGENT_MODEL unless a model object is supplied. **@tool** exposes get_weather to the model. **ask()** invokes the agent and returns a result with a messages list.

The model requests a tool call; the framework executes the Python function; the model receives its result and produces an answer. The system prompt requests exactly one call, but the prompt alone does not enforce that constraint.

Example 1: Known city

The weather data is a fixed dictionary, not an external forecast service.

```
get_weather.invoke({"city": "Tokyo"})
# {"city": "Tokyo", "available": True,
#  "temperature_c": 22, "condition": "sunny"}
```

Example 2: Rainy city

The prompt asks the model to turn rainy conditions into useful advice.

```
get_weather.invoke({"city": "London"})
# available=True, temperature_c=12, condition="rainy"
# Expected answer includes: "Bring an umbrella."
```

Example 3: Unknown city

Only Tokyo and London are in the fixture. Matching is case-sensitive.

```
get_weather.invoke({"city": "Atlantis"})
# {"city": "Atlantis", "available": False}
```

What to remember

Even a live evaluation uses this same fixed weather tool. "Live" refers to real model calls and tracking, not real-time weather.

Pytest and test cases

A test is Python code that checks expected behavior. Pytest discovers test functions, runs them, and reports the results.

Your pytest.ini sets testpaths=tests and pythonpath=., so tests are collected from tests and project modules can be imported. By default, common names include test_*.py files and test_* functions.

Example 1: A passing assertion

assert means the expression must be true.

```
def test_add():
    assert 2 + 3 == 5
# Passes
```

Example 2: A failing assertion

A false assertion fails the test. Unexpected exceptions can also cause failures.

```
def test_wrong_temperature():
    actual_temperature = 22
    assert actual_temperature == 30
# Fails
```

Example 3: One function, three cases

Parametrization creates a separate test item for each supplied value.

```
@pytest.mark.parametrize("city",
                         ["Tokyo", "London", "Atlantis"])
def test_city_name(city):
    assert isinstance(city, str)
# Three collected test cases
```

Tests, fixtures, and hooks

Concept	Question it answers
Test function	What behavior am I checking?
Fixture	What resources or preparation does this test need?
Hook	How should pytest handle this stage of the run?

Your project currently uses hooks in conftest.py and scripted models in offline tests. The term "weather fixture" describes fixed data; it is not a @pytest.fixture function here.

Process grading: AgentEvals

The trajectory is the observable sequence of messages and tool calls. It is not the model's private reasoning. The process grader checks this sequence against a reference.

```
trajectory_evaluator = create_trajectory_match_evaluator(
    trajectory_match_mode="strict",
    tool_args_match_mode="exact",
)
```

strict checks the reference message/tool structure and ordering. **exact** requires exact tool arguments. This matching grader does not need an LLM judge and does not grade the text of the tool response or final answer.

Example 1: Correct path: passes

One weather call for Tokyo, followed by a tool result and final answer, matches the reference.

```
Human -> AI: get_weather(city="Tokyo")
      -> Tool result -> AI answer
```

Example 2: Wrong city: fails

A Tokyo question followed by a London tool call has the wrong argument.

```
Expected: {"city": "Tokyo"}
Actual:   {"city": "London"}
```

Example 3: Missing tool: fails

A guessed answer can contain correct facts but still fail the process requirement.

```
Human -> AI: "Tokyo is sunny and 22 degrees Celsius."
# No get_weather call
```

Example 4: Duplicate call: fails

Two weather calls do not match the exactly-one-call reference.

```
Human -> AI call -> Tool -> AI call -> Tool -> AI answer
```

A factory function creates a reusable callable grader. Importing evaluators.py creates trajectory_evaluator; grading happens when a test later calls it with actual and reference messages.

Answer grading: OpenEvals

The answer grader assesses meaning. It gives a judge model the question, the agent's answer, and a reference answer. The judge can accept correct paraphrases.

```
grader = make_answer_evaluator(model=judge_model)
score = grader(
    inputs=question,
    outputs=answer,
    reference_outputs=expected_answer,
)
```

CORRECTNESS_PROMPT supplies grading instructions. **feedback_key="answer_correctness"** names the score. **model** supplies a model identifier in live tests; **judge** supplies a model object in offline tests. The project explicitly supplies one of these two arguments when creating a grader.

Example 1: Correct paraphrase

Reference: "Tokyo is sunny and 22 degrees Celsius." Actual: "It is 22 C and sunny in Tokyo." Intended verdict: True, because the facts agree.

Example 2: Wrong fact

Reference: Tokyo is 22 C. Actual: Tokyo is 30 C. Intended verdict: False, even if the tool call was correct.

Example 3: Invented unavailable weather

Reference: Atlantis data is unavailable. Actual: Atlantis is sunny at 25 C. Intended verdict: False.

Result shape and limitations

```
{
    "key": "answer_correctness",
    "score": True,
    # A comment may explain the decision.
}
```

These are intended judgments, not guaranteed model outputs. A judge can be wrong. Your tests require score is True, and a serious benchmark should validate judge quality against human ratings. OpenEvals here is LangChain's library, not OpenAI's Evals product.

The evaluator helpers and messages

Three imported classes describe conversation roles: HumanMessage for the user, AIMessage for model output or tool requests, and ToolMessage for tool results.

```
reference_trajectory(question, city, expected_answer)
# Returns:
# HumanMessage(question)
# AIMessage(tool_calls=[get_weather(city)])
# ToolMessage("fixture result")
# AIMessage(expected_answer)
```

The helper describes an expected run; it does not execute get_weather. content="" on the tool-request message means there is no ordinary assistant text. tool_calls is a list of structured requests, each with a name, args, and id.

Example 1: HumanMessage

This is the user's question.

```
HumanMessage(content="Weather in Tokyo?")
```

Example 2: AIMessage requesting a tool

The call identifier links the request to its response within this conversation.

```
AIMessage(content="", tool_calls=[{
    "name": "get_weather", "args": {"city": "Tokyo"},
    "id": "reference_call",
}])
```

Example 3: ToolMessage and final answer

The tool response refers to that request. The final assistant message completes the expected sequence.

```
ToolMessage(content="fixture result",
            tool_call_id="reference_call")
AIMessage(content="Tokyo is sunny and 22 C.")
```

Extracting the final answer: three small examples

```
messages = ["question", "tool request", "tool result", "answer"]
messages[0]    # "question": first item
messages[-1]   # "answer": last item
result["messages"][-1].text  # Last message's text
```

final_answer() assumes the message list is nonempty and ends in the answer. .text handles string content and text blocks. The reference tool response is a placeholder because the process grader does not compare its wording.

07 / LEARNING NOTES

Offline versus live evaluations

Offline checks verify the testing machinery with scripted models. Live checks measure real-model behavior and send experiment data to LangSmith.

Example 1: Offline agent wiring

A scripted model requests Tokyo weather, then returns a scripted sentence. The real agent framework executes the real fixture tool. This checks wiring, not model intelligence.

Example 2: Offline evaluator checks

Wrong-city, missing-tool, and duplicate-call paths must receive False. Scripted judges returning True and False must have those verdicts propagated by OpenEvals.

Example 3: Live end-to-end case

A real agent model answers the London question. AgentEvals checks its path; a judge model grades its answer; LangSmith records the outputs and scores; pytest requires both scores to be True.

The live test sequence

```
Log question and reference answer
    -> Run agent
    -> Extract and log final answer
    -> Grade path and answer
    -> Log feedback
    -> Assert both scores are True
```

@pytest.mark.parametrize supplies three cases. @pytest.mark.live labels them for your skip logic. @pytest.mark.langsmith enables the tracking integration. t.trace_feedback() groups grading activity as feedback rather than application work.

The offline suite was run during our earlier walkthrough: **6 passed, 3 skipped**. Live evaluations were not run in that walkthrough. Three examples are a learning dataset, not evidence of production reliability.

conftest.py and environment settings

Pytest automatically discovers conftest.py. Your file loads settings, registers --live, disables offline tracing, and decides whether live tests can proceed.

```
load_dotenv(Path(__file__).resolve().parents[1] / ".env")
```

__file__ identifies conftest.py. resolve() produces its absolute path. parents[0] is tests; parents[1] is the project root. The / operator joins that root with .env. Loading occurs when the module is imported.

Example 1: Setting comes from .env

If AGENT_MODEL is absent from the process environment, a value in .env becomes available through os.environ. Keep credentials local; do not print them.

Example 2: Existing value takes precedence

If your terminal already defines AGENT_MODEL, load_dotenv() preserves that value by default, even if .env specifies another model.

Example 3: Offline tracing override

Even if .env says LANGSMITH_TRACING=true, pytest_configure explicitly sets it to false for an offline run.

Useful expressions

```
os.environ["AGENT_MODEL"]       # Requires the key to exist
os.environ.get("AGENT_MODEL")   # None if missing
os.environ.get("SETTING", "")   # Empty string if missing
```

Checking that a value exists does not prove it is valid. Live configuration checks cannot establish that credentials work or that your account can access the named model.

Pytest lifecycle: stage-to-hook map

Remember two levels: the whole run has configure, collect, execute, finish; each test has setup, call, teardown. This is the normal lifecycle, not an exhaustive list of hooks.

Stage / hooks	Three examples of work at this stage
Configuration: pytest_addoption; pytest_configure	Register --live; read command-line settings; disable offline tracing.
Session start: pytest_sessionstart	Initialize a session timer; prepare session metadata; record session startup.
Collection: pytest_collection; pytest_itemcollected; pytest_collection_modifyitems; pytest_collection_finish	Discover tests; expand parameterized examples; mark live tests as skipped.
Execution: pytest_runtestloop; pytest_runtest_protocol	Run the Tokyo case; run the London case; run the Atlantis case.
Finish: pytest_sessionfinish; pytest_terminal_summary; pytest_unconfigure	Finish the session; summarize counts; release configuration resources.

Inside each test: three phases

Phase / hook	Three examples
Setup: pytest_runtest_setup	Create a fixture object; initialize test data; acquire a temporary resource.
Call: pytest_runtest_call	Invoke the agent; grade its output; execute assertions.
Teardown: pytest_runtest_teardown	Close a connection; release a temporary resource; run fixture cleanup.

pytest_runtest_logstart and pytest_runtest_logfinish surround a test. After each phase, pytest_runtest_makereport creates a report and pytest_runtest_logreport handles it. A normal test has three phase reports; setup failure or skipping prevents the test body from running.

Hooks run according to pytest's lifecycle, not the order their definitions appear in conftest.py. Ordinary test resources are usually best supplied through fixtures.

Hook 1: pytest_addoption

This configuration-stage hook registers custom command-line options. Pytest supplies parser and calls the hook automatically.

```
def pytest_addoption(parser):
    parser.addoption(
        "--live",
        action="store_true",
        help="Run paid model calls and LangSmith tracking",
    )
```

"--live" names the option. action="store_true" makes it a Boolean switch. help is the description shown by pytest --help. Registering the option does not itself call a model or enable tracking.

Example 1: Flag absent

The default for store_true is False. Other hooks read this and prepare an offline run.

```
pytest -q
# config.getoption("--live") -> False
```

Example 2: Flag present

The flag stores True. Do not supply an extra true/false value.

```
pytest --live -q
# config.getoption("--live") -> True
```

Example 3: Help requested

Pytest prints the option's description. Help displays usage rather than executing the test cases.

```
pytest --help
# --live  Run paid model calls and LangSmith tracking
```

Equivalent explicit default

```
parser.addoption("--live", action="store_true",
                 default=False, help="Enable live evaluations")
```

default=False documents the default explicitly. It does not change your existing behavior.

Hook 2: pytest_configure

This hook runs during configuration, before test collection. Your project uses it to switch off tracing in offline mode.

```
def pytest_configure(config):
    if not config.getoption("--live"):
        os.environ["LANGSMITH_TRACING"] = "false"
```

config is pytest's configuration object. getoption reads an option. not reverses its Boolean meaning, so the body runs when --live is absent.

Example 1: Offline, .env tracing true

Command: pytest -q. The flag is False, not False is True, and the hook changes tracing to "false".

Example 2: Offline, tracing already false

Command: pytest -q. The hook assigns "false" again, preserving the intended offline behavior.

Example 3: Live run

Command: pytest --live -q. The flag is True, not True is False, so this hook leaves the tracing setting as supplied. The later collection hook requires it to be "true".

Boolean translation

```
if not config.getoption("--live"):
    ...

# Read as:
# "If live mode was not requested, do this."
```

The environment assignment affects the current process and children that inherit its environment. It does not rewrite your .env file.

Hook 3: pytest_collection_modifyitems

This collection-stage hook receives config and items after tests are discovered. Each item is one executable test case, including each parameterized example.

```
if config.getoption("--live"):
    # Check required settings and tracing.
else:
    for item in items:
        if item.get_closest_marker("live"):
            item.add_marker(pytest.mark.skip(
                reason="Use --live with configured credentials"
            ))
```

Example 1: Offline: skip live items

For pytest -q, each item carrying the live marker receives a skip marker. Offline items run normally. A marker is metadata; this hook gives live its skip behavior.

Example 2: Live: missing setting

If JUDGE_MODEL is absent or empty, the hook raises pytest.UsageError with a configuration message. This stops the run before test execution; it is not an agent failure.

Example 3: Live: settings present

With all four required settings and LANGSMITH_TRACING=true, this gate allows execution. Network or authentication errors could still occur later.

The missing-settings expression

```
required = ["OPENAI_API_KEY", "LANGSMITH_API_KEY",
            "AGENT_MODEL", "JUDGE_MODEL"]
missing = [name for name in required
           if not os.environ.get(name)]
# Example missing list: ["AGENT_MODEL", "JUDGE_MODEL"]
", ".join(missing)  # "AGENT_MODEL, JUDGE_MODEL"
```

The tracing check applies .lower() to accept "true", "TRUE", or "True". It does not remove whitespace. Your gate runs whenever --live is present, even if you select only offline tests.

Command-line actions: Boolean or value

action configures how a command-line option is parsed. These are alternative designs you could add to pytest_addoption; only --live currently exists in your project.

store_true: presence means True

```
parser.addoption("--live", action="store_true")
pytest                    # False
pytest --live             # True
pytest --live -q           # True; -q changes verbosity
```

store_false: presence means False

```
parser.addoption("--no-live", action="store_false",
                 dest="live")
pytest                    # live=True
pytest --no-live          # live=False
pytest --no-live -q       # live=False
```

This reverses the switch and is only an illustrative design. A live-by-default mode would change the project's current offline default.

store: save a supplied value

```
parser.addoption("--mode", action="store",
                 choices=["offline", "live"], default="offline")
pytest                    # mode="offline"
pytest --mode live        # mode="live"
pytest --mode offline     # mode="offline"
```

choices restricts accepted values. Using --mode live would require updating the other hooks to check config.getoption("--mode") == "live" instead of checking --live.

Reading values

```
config.getoption("--live")   # Current project
config.getoption("live")     # dest from store_false example
config.getoption("--mode")   # Alternative store example
```

Command-line actions: lists and counts

Use append for repeated values and count for repeated switches. These examples are hypothetical additions; the project does not currently define --city or --detail.

append: collect supplied values

```
parser.addoption("--city", action="append", default=[])
```

Example 1: No supplied values

Default result is an empty list.

```
pytest
# config.getoption("--city") -> []
```

Example 2: One supplied value

One occurrence adds one list entry.

```
pytest --city Tokyo
# ["Tokyo"]
```

Example 3: Two supplied values

Repeated occurrences preserve the supplied values.

```
pytest --city Tokyo --city London
# ["Tokyo", "London"]
```

count: count occurrences

```
parser.addoption("--detail", action="count", default=0)
pytest                    # detail=0
pytest --detail           # detail=1
pytest --detail --detail  # detail=2
```

Reading a city list or count does not automatically select cities or change verbosity. You must write code that uses the parsed value. Choose an action based on the kind of input you want: Boolean, one value, list, or number of occurrences.

Terminal commands and help

Open VS Code: Terminal > New Terminal. Ensure the terminal is in weather-evals, then activate your existing virtual environment.

```
source .venv/bin/activate
```

Help: three ways to inspect it

```
pytest --help
# All available options

pytest --help | rg -- "--live"
# Only lines containing --live (requires ripgrep)

pytest --help | grep -- "--live"
# Same filter using grep
```

The pipe | passes help text to the filter. The -- tells rg or grep that following text is a search pattern, even though it begins with hyphens. The correct flag is --help; --elf is a typo. The help description documents behavior but does not implement it.

Run tests: three common commands

```
pytest -q
# Offline tests run; live cases are skipped

pytest -q tests/test_offline.py
# Run only the offline test file

pytest --live -q tests/test_live_evals.py
# Run live evaluations; configured .env required
```

-q requests quieter output. Live evaluations make paid model calls and record data in LangSmith. pytest --live -q without a file selects both offline and live tests.

If pytest is not found

```
.venv/bin/python -m pytest --help
.venv/bin/python -m pytest -q
```

These commands use the project environment explicitly, without requiring activation. The packages must already be installed in that environment.

Review, practice, and references

Use these three worked scenarios to connect the pieces. Follow the path from terminal option to test execution and grading.

Example 1: Explain an offline run

pytest -q stores --live=False. pytest_configure disables tracing. Collection marks live cases as skipped. Offline scripted checks run. The earlier walkthrough reported 6 passed and 3 skipped.

Example 2: Explain a live configuration error

pytest --live stores True. If JUDGE_MODEL is missing, collection raises UsageError. No agent answer is graded because the run stops before executing test bodies.

Example 3: Explain an answer-only failure

The agent calls get_weather(city="Tokyo") once, but answers "30 C." Its path can pass, while the intended answer judgment is False. The final answer assertion then fails the test.

Remember the responsibilities

agent.py: application behavior. **evaluators.py:** reusable grading tools. **test_live_evals.py:** run, log, grade, assert. **test_offline.py:** check wiring with scripted models. **conftest.py:** control test-run setup and eligibility.

Project reading order

Start with agent.py, then evaluators.py, then test_live_evals.py. Read conftest.py to understand the mode switch. Finish with test_offline.py to see how the machinery is verified without real model calls.

Primary references

pytest official hook reference: lifecycle and runtest protocol. This reference was consulted during our discussion.

Further reading from the project README: pytest getting started; LangSmith pytest integration; trajectory evaluations; OpenEvals Python.

Code examples labeled as alternative designs are teaching examples, not changes to your project. No live evaluation was executed to create this guide.